Multi-Solver Coupling Strategies

Design note and experimental API

Newton coupling notes

2026-04-23

1 Scope

This note combines the coupling design background with the implemented experimental Python API. The mathematical sections use a common notation to compare several ways to run two solvers on one simulation step. The API section describes the current experimental surface exported from `newton.solvers.coupled_experimental`; that namespace and the numerical defaults may still change while the feature is refined. Sections that discuss implicit-interface ADMM remain design rationale unless they are explicitly tied to the implemented public API.

The variants considered below are:

- collider exchange,
- one-way coupling,
- proxy bodies / virtual inertia,
- linearized ADMM,
- implicit-interface ADMM.

They all start from the same velocity-level interface problem. The main differences are where the interface law is solved, how action-reaction consistency is obtained, and what API each scheme requires from the sub-solvers.

Notation map from earlier notes: (M_1, M_2, H_1, H_2) are written here as (M_a, M_b, J_a, J_b) for solvers a and b. The Delassus operator is denoted by D. The ADMM contact-space preconditioner is denoted by P.

2 Common interface problem

Let $\boldsymbol{v}_a \in \mathbb{R}^{n_a}$ and $\boldsymbol{v}_b \in \mathbb{R}^{n_b}$ be the generalized velocities owned by solvers a and b during one time step. The matrices M_a and M_b denote the step inertia or the local quadratic model used by each sub-solver. The vectors $\boldsymbol{f}_a, \boldsymbol{f}_b$ collect explicit forces, controls, and internal terms after the local time-integration linearization.

The interface relative velocity is

$$\boldsymbol{u} = J_a \boldsymbol{v}_a + J_b \boldsymbol{v}_b, \tag{1}$$

where J_a and J_b are contact or attachment Jacobians. They may represent normal contact velocities, tangential slip velocities, rigid attachment velocities, or other low-dimensional interface quantities. Bias velocities and stabilization terms can be folded into a target $\bar{\boldsymbol{u}}$.

For a hard bilateral velocity constraint $\boldsymbol{u} = \bar{\boldsymbol{u}}$, the monolithic coupled solve is

$$\begin{bmatrix} M_a & 0 & -J_a^\top \\ 0 & M_b & -J_b^\top \\ -J_a & -J_b & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{v}_a \\ \boldsymbol{v}_b \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} \boldsymbol{f}_a \\ \boldsymbol{f}_b \\ -\bar{\boldsymbol{u}} \end{bmatrix}. \tag{2}$$

The multiplier $\boldsymbol{\lambda}$ is the interface impulse in the sign convention

$$M_a \boldsymbol{v}_a = \boldsymbol{f}_a + J_a^\top \boldsymbol{\lambda}, \qquad M_b \boldsymbol{v}_b = \boldsymbol{f}_b + J_b^\top \boldsymbol{\lambda}.$$

For unilateral contact, the third row is replaced by a contact law. In the frictionless normal direction this is the Signorini condition

$$0 \le u_n \perp \lambda_n \ge 0.$$

Friction modifies the local contact law but not the ownership split.

Delassus form. Eliminating $\boldsymbol{v}_a$ and $\boldsymbol{v}_b$ gives the contact-space relation

$$\boldsymbol{u} = D\boldsymbol{\lambda} + \boldsymbol{u}^0, \qquad D = J_a M_a^{-1} J_a^\top + J_b M_b^{-1} J_b^\top, \qquad \boldsymbol{u}^0 = J_a M_a^{-1} \boldsymbol{f}_a + J_b M_b^{-1} \boldsymbol{f}_b. \tag{3}$$

D is the Delassus operator, or contact-space inverse inertia. Its inverse $K_{\text{eff}} = D^{-1}$ is the effective mass when it exists.

Finite stiffness form. For a finite bilateral stiffness κ with energy $E_c(\boldsymbol{u}) = \frac{\kappa}{2}\|\boldsymbol{u} - \bar{\boldsymbol{u}}\|^2$, the monolithic velocity solve can also be written as

$$\begin{bmatrix} M_a + \kappa J_a^\top J_a & \kappa J_a^\top J_b \\ \kappa J_b^\top J_a & M_b + \kappa J_b^\top J_b \end{bmatrix} \begin{bmatrix} \boldsymbol{v}_a \\ \boldsymbol{v}_b \end{bmatrix} = \begin{bmatrix} \boldsymbol{f}_a + \kappa J_a^\top \bar{\boldsymbol{u}} \\ \boldsymbol{f}_b + \kappa J_b^\top \bar{\boldsymbol{u}} \end{bmatrix}. \tag{4}$$

All variants below avoid directly assembling this full coupled system.

3 Coupling variants

3.1 Collider exchange

In collider exchange, each sub-solver sees the other side as kinematic collision geometry. The two sub-solvers advance independently, each with its own contact solve. This is the least intrusive scheme: it requires only pose and velocity updates for the foreign collision geometry.

The approximation is that the two sides do not solve one shared interface impulse. Consequently, the impulses computed by the two contact solvers need not be equal and opposite. The combined interface complementarity problem is also not enforced as one problem, and either side can receive kinematic boundary velocities that make its local contact problem inconsistent. The method is mainly justified when the time step is small or when the foreign side is intentionally treated as prescribed scenery.

3.2 One-way coupling

One-way coupling is the limiting case where side a is prescribed or much heavier than side b. Setting M_a^{-1} to zero in (3), side a first advances to

$$\boldsymbol{v}_a^\star = M_a^{-1} \boldsymbol{f}_a,$$

and side b solves

$$\boldsymbol{u} = J_a \boldsymbol{v}_a^\star + J_b M_b^{-1} \boldsymbol{f}_b + J_b M_b^{-1} J_b^\top \boldsymbol{\lambda}. \tag{5}$$

The interface impulse is scaled by side b's effective inertia. If $M_b \ll M_a$, applying $J_a^\top \boldsymbol{\lambda}$ back to side a produces a negligible change, so the approximation is consistent with the mass ratio. If the mass ratio assumption is not valid, the scheme becomes a modeling choice rather than a controlled approximation.

3.3 Proxy bodies / virtual inertia

In the proxy-body variant, objects owned by side a are represented in side b by replica degrees of freedom with an approximate inertia $\widehat{M}_a$. The proxy inertia is chosen to be easy for side b to solve, for example independent rigid-body blocks or a diagonal approximation. A scalar relaxation factor can be used to make the proxies lighter or heavier in the destination solve.

Suppose an impulse $J_{a,k}^\top \boldsymbol{\lambda}^k$ was applied to side a in the previous outer iteration or previous time step, producing

$$\boldsymbol{v}_a^k = M_a^{-1}\left(\boldsymbol{f}_a + J_{a,k}^\top \boldsymbol{\lambda}^k\right). \tag{6}$$

The proxy solve uses the virtual-inertia approximation

$$\boldsymbol{v}_a \approx \boldsymbol{v}_a^k + \widehat{M}_a^{-1}\left(J_a^\top \boldsymbol{\lambda} - J_{a,k}^\top \boldsymbol{\lambda}^k\right), \tag{7}$$

or equivalently

$$\widehat{M}_a(\boldsymbol{v}_a - \boldsymbol{v}_a^k) = J_a^\top \boldsymbol{\lambda} - J_{a,k}^\top \boldsymbol{\lambda}^k. \tag{8}$$

The destination solver then solves

$$\begin{bmatrix} \widehat{M}_a & 0 & -J_a^\top \\ 0 & M_b & -J_b^\top \\ -J_a & -J_b & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{v}_a \\ \boldsymbol{v}_b \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} \widehat{M}_a \boldsymbol{v}_a^k - J_{a,k}^\top \boldsymbol{\lambda}^k \\ \boldsymbol{f}_b \\ -\bar{\boldsymbol{u}} \end{bmatrix}. \tag{9}$$

The subtraction term $-J_{a,k}^\top \boldsymbol{\lambda}^k$ prevents the previously applied impulse from being counted again. The resulting impulse can be applied to side a in the next time step, or in an outer fixed-point iteration.

For frozen Jacobians and bilateral constraints, define the proxy Delassus

$$\widehat{D} = J_a \widehat{M}_a^{-1} J_a^\top + J_b M_b^{-1} J_b^\top.$$

The fixed-point error propagation for the impulse contains the matrix

$$\widehat{D}^{-1} J_a \left(\widehat{M}_a^{-1} - M_a^{-1}\right) J_a^\top. \tag{10}$$

A spectral radius below one gives a local contraction. This condition is favored when $\widehat{M}_a$ is close to M_a in the interface directions, when side b is light relative to side a, or when the proxy inertia is relaxed enough to damp the update.

For a scalar interface with $J_a = 1$, write the virtual-inertia scale as $s = \widehat{M}_a / M_a$ and the mass ratio as $q = M_a / M_b$. Then

$$r(s,q) = \frac{\widehat{M}_a^{-1} - M_a^{-1}}{\widehat{M}_a^{-1} + M_b^{-1}} = \frac{1/s - 1}{1/s + q} = \frac{1-s}{1+qs}. \tag{11}$$

Thus $s = 1$ gives exact inertia matching and $r = 0$. For $0 < s < 1$ the scalar map is contractive for any positive q, but the limit $s \to 0$ approaches $r \to 1$ from below, so very light proxies are only weakly contractive. For $s > 1$ the fixed-point factor changes sign, and it becomes non-contractive when the overscaled proxy is large enough relative to the side-b inertia.

The sign and magnitude of Eq. (11) are useful as a two-parameter diagnostic over s and q. The repository keeps the executable visualization in `example_proxy_coupling_convergence.py`; the generated plot is intentionally not checked into the documentation image gallery while the coupled examples remain experimental.

When the true source-side effective inertia is not available, the proxy mass scale should be treated as a relaxation parameter rather than as a precise physical mass. A practical policy is:

1. build the cheapest available estimate M_{est} of source inertia at the interface: particle mass for particles, directional rigid effective mass $1/(J_a M_a^{-1} J_a^\top)$ when available, translational body mass otherwise, or a local particle-stencil mass for cloth and soft bodies;
2. initialize $\widehat{M}_a = sM_{\text{est}}$ with $s \in [0.5, 1]$, using the lower end when the estimate is crude and the upper end when the estimate is trusted;
3. measure feedback convergence from harvested forces or impulses, for example $e_k = \|\boldsymbol{\lambda}_k - \boldsymbol{\lambda}_{k-1}\|$ or, in force form, $e_k = \|\mathbf{f}_k - \mathbf{f}_{k-1}\|$;
4. adapt the scale from the observed residual ratio $\hat{r} = e_k / \max(e_{k-1}, \epsilon)$: reduce s when $\hat{r}$ is large or the feedback oscillates, keep it when $\hat{r}$ is moderate, and increase s toward one when $\hat{r}$ is small;
5. clamp the result, for example $s \in [0.05, 1]$, so the proxy stays conservative and avoids overscaled-inertia sign flips unless a solver-specific effective mass estimate justifies $s > 1$.

This requires no new mandatory solver interface if adaptation happens between top-level steps: the shared coupler can use its existing harvested feedback buffers, update the stored proxy scale, and rescale the destination `ModelView` mass overlay before the next step. A runtime API to mutate proxy mass scales, or a small `notify_model_changed` path for view-local mass overlays, would make this implementation cleaner and avoid rebuilding the coupled solver. Per-interface effective-mass hooks would still be useful for better initialization, but they are an optimization rather than a prerequisite for adaptive proxy relaxation.

3.4 Linearized ADMM

ADMM introduces an explicit interface variable and keeps the ownership of physical degrees of freedom disjoint. The coupling law is represented by an interface energy or indicator $E_c(\boldsymbol{u})$:

- hard bilateral attachment: indicator of $\boldsymbol{u} = \bar{\boldsymbol{u}}$;
- compliant attachment: $E_c(\boldsymbol{u}) = \frac{\kappa}{2}\|\boldsymbol{u} - \bar{\boldsymbol{u}}\|^2$;
- frictionless unilateral contact: indicator of the admissible normal velocity set;
- frictional contact: local maximum-dissipation Coulomb velocity solve.

The split problem is

$$\min_{\boldsymbol{v}_a, \boldsymbol{v}_b, \boldsymbol{u}} \; E_a(\boldsymbol{v}_a) + E_b(\boldsymbol{v}_b) + E_c(\boldsymbol{u}) \quad \text{s.t.} \quad \boldsymbol{u} = J_a \boldsymbol{v}_a + J_b \boldsymbol{v}_b. \tag{12}$$

E_a and E_b are the sub-solvers' per-step objectives, including their internal forces and contacts.

Let

$$\boldsymbol{r}(\boldsymbol{v}_a, \boldsymbol{v}_b, \boldsymbol{u}) = \boldsymbol{u} - J_a \boldsymbol{v}_a - J_b \boldsymbol{v}_b.$$

Using the physical interface impulse $\boldsymbol{\lambda}$ as the dual variable and a symmetric positive-definite contact-space preconditioner P, the augmented Lagrangian is

$$\mathcal{L}_\rho(\boldsymbol{v}_a, \boldsymbol{v}_b, \boldsymbol{u}, \boldsymbol{\lambda}) = E_a(\boldsymbol{v}_a) + E_b(\boldsymbol{v}_b) + E_c(\boldsymbol{u}) + \langle \boldsymbol{\lambda}, \boldsymbol{r} \rangle + \frac{\rho}{2}\|\boldsymbol{r}\|_P^2, \qquad \|x\|_P^2 = x^\top P x. \tag{13}$$

A natural choice is $P \approx K_{\text{eff}} = D^{-1}$, so the penalty is scaled by contact-space effective mass. Equivalently, one may write $P = W^\top W$ with $W \approx K_{\text{eff}}^{1/2}$.

A direct ADMM velocity update would add the Hessian $\rho J_a^\top P J_a$ to solver a and $\rho J_b^\top P J_b$ to solver b. Linearized ADMM avoids that solver-internal Hessian by evaluating the penalty gradient at the present iterate and adding a local proximal term. With

$$\boldsymbol{r}^k = \boldsymbol{u}^k - J_a \boldsymbol{v}_a^k - J_b \boldsymbol{v}_b^k, \qquad \boldsymbol{g}^k = \boldsymbol{\lambda}^k + \rho P \boldsymbol{r}^k,$$

the parallel velocity sweep is

$$\boldsymbol{v}_a^{k+1} = \arg\min_{\boldsymbol{v}_a} \; E_a(\boldsymbol{v}_a) - \langle J_a^\top \boldsymbol{g}^k,\, \boldsymbol{v}_a\rangle + \frac{\alpha_a}{2}\|\boldsymbol{v}_a - \boldsymbol{v}_a^k\|_{M_a}^2, \tag{14}$$

$$\boldsymbol{v}_b^{k+1} = \arg\min_{\boldsymbol{v}_b} \; E_b(\boldsymbol{v}_b) - \langle J_b^\top \boldsymbol{g}^k,\, \boldsymbol{v}_b\rangle + \frac{\alpha_b}{2}\|\boldsymbol{v}_b - \boldsymbol{v}_b^k\|_{M_b}^2. \tag{15}$$

A sufficient convexity condition is

$$\alpha_a M_a \succeq \rho\, J_a^\top P J_a, \qquad \alpha_b M_b \succeq \rho\, J_b^\top P J_b.$$

After the velocity sweep, the interface variable is updated by the small separable problem

$$\boldsymbol{u}^{k+1} = \arg\min_{\boldsymbol{u}} \; E_c(\boldsymbol{u}) + \langle \boldsymbol{\lambda}^k,\, \boldsymbol{u}\rangle + \frac{\rho}{2}\|\boldsymbol{u} - J_a\boldsymbol{v}_a^{k+1} - J_b\boldsymbol{v}_b^{k+1}\|_P^2, \tag{16}$$

followed by the dual update

$$\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}^k + \rho P\left(\boldsymbol{u}^{k+1} - J_a\boldsymbol{v}_a^{k+1} - J_b\boldsymbol{v}_b^{k+1}\right). \tag{17}$$

For a damped quadratic attachment,

$$E_c(\boldsymbol{u}) = \frac{\kappa}{2}\|\boldsymbol{u} - \bar{\boldsymbol{u}}\|^2 + \frac{\eta}{2}\|\boldsymbol{u}\|^2,$$

the local update remains diagonal per row:

$$\boldsymbol{u}^{k+1} = \frac{\rho W^2(J_a\boldsymbol{v}_a^{k+1} + J_b\boldsymbol{v}_b^{k+1}) + \kappa\bar{\boldsymbol{u}} - W\boldsymbol{\lambda}^k}{\kappa + \eta + \rho W^2}.$$

The damping coefficient η adds resistance to relative interface velocity without changing the Baumgarte or spring target $\bar{\boldsymbol{u}}$. For model-joint box friction with per-axis limits $\boldsymbol{\tau}$, the same local problem uses the support function $\sum_i \tau_i|u_i|$ and reduces to a component-wise soft-threshold of the relative angular velocity in the joint frame. This is the box analogue of the contact maximum-dissipation solve. The interface update is a tiny dense solve for quadratic attachments, a clamp for a frictionless normal contact, or a local maximum-dissipation Coulomb solve for frictional contact. With isotropic contact weight W and

$$\boldsymbol{u}^\star = J_a\boldsymbol{v}_a^{k+1} + J_b\boldsymbol{v}_b^{k+1} - \frac{\boldsymbol{\lambda}^k}{\rho W}, \qquad \bar{\boldsymbol{u}} = \boldsymbol{u}^\star - u_{\min}\boldsymbol{n},$$

the implemented Coulomb update applies the Daviet local map in the normal/tangent frame:

$$\boldsymbol{u}^{k+1} = u_{\min}\boldsymbol{n} + \begin{cases} \bar{\boldsymbol{u}}, & \bar{u}_n \geq 0, \\ 0, & \|\bar{\boldsymbol{u}}_t\| \leq -\mu\bar{u}_n, \\ (1 + \mu\bar{u}_n/\|\bar{\boldsymbol{u}}_t\|)\,\bar{\boldsymbol{u}}_t, & \text{otherwise.} \end{cases}$$

The middle branch is sticking. The last branch is sliding: the normal velocity is at the unilateral bound and the dual update places the tangential contact force on the Coulomb boundary, opposite the slip direction. This solves the Coulomb maximum-dissipation law rather than projecting an already-computed force onto a cone.

3.5 Implicit-interface ADMM

The full ADMM variant keeps the quadratic penalty inside each sub-solver instead of linearizing it. The update for side a contains the term

$$\frac{\rho}{2}\|\boldsymbol{u}^k - J_a\boldsymbol{v}_a - J_b\boldsymbol{v}_b^k\|_P^2,$$

and side b receives the analogous term. This removes the proximal tuning parameters α_a, α_b and usually reduces the number of ADMM iterations. The cost is a stronger sub-solver API: each solver must accept implicit point attachments, implicit springs, or another representation of $J_a^\top P J_a$ within its own solve.

This variant is closest to the monolithic finite-stiffness system (4), while still allowing the two velocity blocks to be minimized by their native solvers.

4 API additions relative to upstream/main

The upstream/main baseline provides a generic

```
SolverBase.step(state_in, state_out, control, contacts, dt)
```

interface, external force arrays on `State` such as `body_f` and `particle_f`, model-change notification via `notify_model_changed`, and model/state attribute metadata. It does not provide a public coupled-solver orchestration layer, model partition views, proxy-body metadata, interface Jacobian actions, or a standard solver API for effective mass and implicit interface terms.

4.1 Common APIs

Most variants benefit from the following additions:

- **Model partitioning.** A way to define solver ownership for bodies, particles, shapes, and joints. This should include collision ownership: internal pairs owned by one sub-solver, cross-owner pairs retained for interface coupling, and static geometry assigned explicitly.
- **Model views.** A lightweight view or subset object that presents one sub-solver with only the attributes it owns, plus controlled overrides such as mass, inertia, collision masks, or kinematic state. This avoids cloning whole models for every variant.
- **State distribution and reconciliation.** Utilities to scatter a global state into per-solver states and gather the owned outputs back. The API must define which side owns overlapping quantities and how joint, body, and particle arrays are merged.
- **External-force contract.** A solver-level guarantee that `body_f` and `particle_f` are interpreted as additive external loads for that step, and a clear rule for whether solvers read, modify, or preserve those arrays.
- **Coupling constraint data.** Internal row records derived from model joints or collision detection: endpoint type and index, local anchor, normal/tangent basis, target velocity or bias, compliance, friction, and owner side.
- **Jacobian actions.** Kernels or methods for evaluating Jv and applying $J^\top f$ for rigid bodies, particles, and mixed endpoint pairs. Materializing J should not be required.
- **Effective-mass estimates.** Optional methods to evaluate diagonal or block-diagonal approximations of $JM^{-1}J^\top$ for contact-space scaling.

4.2 APIs for collider exchange and one-way coupling

These schemes need the smallest API surface:

- update kinematic collision geometry from another solver's state;
- filter collision pairs so that each sub-solver receives the intended collider set;
- optionally accumulate or report reaction forces if one-way feedback is desired.

They do not require a shared interface dual variable or repeated in-step iterations.

4.3 APIs for proxy / virtual-inertia coupling

Proxy coupling requires additional support:

- create or expose replica bodies or particles in the destination model without treating them as independently owned physical outputs;
- mark proxy bodies and particles as coupling metadata rather than ordinary dynamic or kinematic entities;
- override proxy mass and inertia in a solver view, including block rigid inertia or diagonal approximations;
- sync proxy poses and velocities from the source side before the destination solve;
- remove previously applied impulses, gravity, or other loads that would otherwise be counted twice on proxy degrees of freedom;
- report contact impulses or accumulated wrenches on proxies after the destination solve so that the source side can receive the reaction;
- optionally run an outer fixed-point loop with convergence metrics on interface velocity and impulse.

4.4 APIs for linearized ADMM

Linearized ADMM can avoid arbitrary coupling Hessians, but it needs iteration-level control:

- repeat sub-solver steps from the same begin-of-step state while changing only external coupling forces and proximal targets;
- apply $J^\top \boldsymbol{g}$ as body and particle forces before each ADMM velocity sweep;
- express the proximal term $\frac{\alpha}{2}\|v - v^k\|_M^2$ either as a first-class solver option or through mass scaling plus a rewritten pre-step velocity;
- keep warm-started interface variables $\boldsymbol{u}$ and $\boldsymbol{\lambda}$ across time steps;
- expose residuals, stopping criteria, and graph-capture-safe iteration buffers.

4.5 APIs for implicit-interface ADMM

The implicit variant needs the strongest solver interface:

- add per-interface implicit springs or attachments to a solver step;
- represent local Hessian contributions such as $\rho J^\top P J$ without forcing callers to assemble solver-internal matrices;
- expose enough effective-mass information to tune ρ and P robustly;
- preserve the same interface data model as linearized ADMM so the two ADMM variants can share contact detection, Jacobian actions, and dual updates.

5 Implemented public Python API

The implemented public API lives in `newton.solvers.coupled_experimental`; examples and user code import the experimental coupling symbols from that namespace. The implementation modules remain under `newton._src`.

The recommended import style is direct import from the experimental namespace:

```
from newton.solvers.coupled_experimental import (
    CouplingInterface,
    ModelView,
    SolverAdmmCoupled,
    SolverCoupled,
    SolverProxyCoupled,
)
```

The implemented split uses a shared coupled-solver base plus algorithm-specific derived couplers. The shared coupler owns state partitioning, `ModelView` construction, state reconciliation, per-entry substeps, and generic coupling hook helpers. Lagged-impulse proxy coupling lives in `SolverProxyCoupled`; linearized ADMM lives in `SolverAdmmCoupled`.

The currently exported `coupled_experimental` symbols are:

- `CouplingInterface`;
- `ModelView` and `SolverCoupled`;
- `SolverProxyCoupled` and `SolverAdmmCoupled`.

5.1 Shared coupled-solver API

```
from collections.abc import Callable, Sequence
from dataclasses import dataclass

import newton as nt
import warp as wp
from newton.solvers import SolverBase


class SolverCoupled(SolverBase, CouplingInterface):
    @dataclass(frozen=True)
    class Entry:
        name: str
        solver: Callable[[ModelView], SolverBase]
        bodies: Sequence[int] = ()
        particles: Sequence[int] = ()
        joints: Sequence[int] = ()
        shapes: Sequence[int] = ()
        configure_view: Callable[[ModelView], None] | None = None
        substeps: int = 1
        in_place: bool = False

    def __init__(
        self,
```

```
        model: nt.Model,
        entries: Sequence["SolverCoupled.Entry"],
        coupling: object | None = None,
    ) -> None: ...

    def solver(self, name: str) -> SolverBase: ...

    def view(self, name: str) -> ModelView: ...


class SolverProxyCoupled(SolverCoupled):
    @dataclass(frozen=True)
    class Proxy:
        source: str
        destination: str
        bodies: Sequence[int] = ()
        proxy_bodies: Sequence[int] | None = None
        mass_scale: float = 1.0
        mode: str = "lagged"
        particles: Sequence[int] = ()
        proxy_particles: Sequence[int] | None = None
        collision_pipeline: Callable[[ModelView], object | None] | None = None
        collide_interval: int | None = None

    @dataclass(frozen=True)
    class Config:
        proxies: Sequence["SolverProxyCoupled.Proxy"]
        iterations: int = 1

    def __init__(
        self,
        model: nt.Model,
        entries: Sequence["SolverCoupled.Entry"],
        coupling: "SolverProxyCoupled.Config",
    ) -> None: ...

    def get_proxy_contacts(
        self,
        source: str,
        destination: str,
    ) -> nt.Contacts | None: ...
```

`SolverCoupled.Entry` describes ownership, sub-solver construction, and optional entry-local time substeps. The solver callable is constructed as `solver(view)` with the entry `ModelView`; bind extra solver options in the callable itself, for example `lambda view: SolverVBD(model=view, iterations=20)`. `substeps` splits that entry's solve into equal sub-intervals. `in_place=True` aliases the entry's input and output states and calls `solver.step(state_0, state_0, ...)`; use it only for solvers that explicitly support same-object input/output stepping. In-place entries currently

require `substeps=1`, and lagged proxy coupling rejects in-place source entries because the source begin pose must remain available. `SolverCoupled` builds one `ModelView` per entry, disables non-owned dynamic bodies, particles, and joints in that view when ownership is specified, and reconciles only owned body, particle, and joint outputs back into the caller's `state_out`. The constructor only accepts explicit `entries`; older map-based prototype construction paths have been removed. Derived couplers may keep non-owned endpoints dynamic in a view and mark them as proxies before sub-solver construction. Proxy coupling is enabled by constructing `SolverProxyCoupled` with a `Config` object.

5.2 ModelView

```
class ModelView:
    def __init__(self, parent: nt.Model, name: str) -> None: ...

    @property
    def name(self) -> str: ...

    @property
    def parent(self) -> nt.Model: ...

    @property
    def overrides(self) -> dict[str, object]: ...

    def state(self, requires_grad: bool | None = None) -> nt.State: ...

    def disable_body_dynamics(self, body_indices: wp.array[int]) -> None: ...

    def scale_body_mass(self, body_indices: wp.array[int], factor: float) -> None: ...

    def set_body_mass(
        self,
        body_indices: wp.array[int],
        body_mass: wp.array[float],
    ) -> None: ...

    def set_body_inertial_properties(
        self,
        body_indices: wp.array[int],
        body_mass: wp.array[float],
        body_inertia: wp.array[wp.mat33],
    ) -> None: ...

    def mark_proxy_bodies(self, body_indices: wp.array[int]) -> None: ...

    def mark_proxy_particles(self, particle_indices: wp.array[int]) -> None: ...

    def deactivate_particles(self, particle_indices: wp.array[int]) -> None: ...
```

```
    def disable_joints(self, joint_indices: wp.array[int]) -> None: ...

    def zero_particle_mass(self, particle_indices: wp.array[int]) -> None: ...

    def scale_particle_mass(
        self,
        factor: float,
        particle_indices: wp.array[int] | None = None,
    ) -> None: ...

    def set_particle_mass(
        self,
        particle_indices: wp.array[int],
        particle_mass: wp.array[float],
    ) -> None: ...
```

`ModelView` has copy-on-write overlay semantics for its explicit mutator methods. Reads fall through to the parent model until a mutator clones the affected array into the view's overrides. The parent model is not mutated by `disable_body_dynamics`, `scale_body_mass`, `set_body_mass`, `set_body_inertial_properties`, `mark_proxy_bodies`, `mark_proxy_particles`, `deactivate_particles`, `disable_joints`, `zero_particle_mass`, `scale_particle_mass`, or `set_particle_mass`. Direct writes through a returned Warp array are not intercepted, so callers should use the mutator methods for view-local edits.

5.3 Proxy / virtual-inertia coupling

```
solver = SolverProxyCoupled(
    model,
    entries=[rigid_entry, soft_entry],
    coupling=SolverProxyCoupled.Config(
        proxies=[
            SolverProxyCoupled.Proxy(
                source="rigid",
                destination="soft",
                bodies=rigid_body_ids,
                proxy_bodies=rigid_proxy_body_ids,
                mass_scale=0.25,
                mode="lagged",
                collision_pipeline=lambda model: CollisionPipeline(model),
                collide_interval=1,
            )
        ]
    ),
)
```

`proxy_bodies=None` maps each source body to the same body index in the destination view; otherwise `proxy_bodies` must have the same length as `bodies`. `proxy_particles=None` does the same for source particles. `mass_scale` scales proxy body mass/inertia and proxy particle mass in the destination view. `mode` accepts the strings `"lagged"` and `"staggered"`. `"lagged"` syncs

the source begin-of-step state and end velocity, then rewinds destination proxy velocities by the previously applied feedback wrench or particle force, public force input, and gravity unless the destination solver provides a custom body or particle proxy velocity-rewind hook. `"staggered"` syncs the source end state and end velocity and skips the generic rewind. If `collision_pipeline` is supplied, the proxy coupler calls the factory with the destination `ModelView`. If the factory returns `None`, no proxy-local collision pipeline is installed and the destination solve receives the outer-level contacts supplied to `step()`. Otherwise the coupler allocates a persistent contacts buffer from the returned pipeline and refreshes those contacts before the destination solve every `collide_interval` proxy passes. A `None` interval means every proxy pass when a custom pipeline is supplied; with no custom pipeline, the coupler keeps the previous behavior and uses the contacts supplied to `step()`. `get_proxy_contacts()` exposes the internal contact buffer for diagnostics and rendering; it returns `None` when no proxy-local pipeline is installed. Destination solvers may also override the hook methods associated with `BODY_PROXY_HARVEST` or `PARTICLE_PROXY_HARVEST` to replace generic momentum harvesting; otherwise the shared coupler estimates feedback from proxy momentum change. Custom harvest hooks receive the prepared destination input state, the destination output state, and the contacts used for the proxy solve, so contact-native solvers can report feedback from final contact forces rather than from aggregate solver forces or arbitrary momentum changes. VBD uses this path to harvest rigid-rigid contact forces through its per-contact collector and body-particle contact forces through a matching soft-contact reducer. MPM has two particle-proxy paths. Pure proxy particles stay active in the MPM transfer flags, so they participate in the normal P2G/G2P momentum transfer, but are masked out of material volume, stress, strain, and constitutive updates. Their feedback is harvested from the transfer-induced momentum change with gravity removed from the lagged velocity rewind and from the final force. Deformable collider particles registered through `collider_particle_ids` remain inactive in both transfer and material terms and use the private collider impulse collector instead. The `iterations` field repeats this proxy relaxation pass over the same top-level time interval. The implemented generic proxy loop currently supports at most two solver entries; larger proxy-coupled configurations should use a more explicit scheduling layer. Within that two-entry limit, the loop groups body and particle mappings by `(source, destination)` and performs one source step and one destination step per solver pair per proxy iteration. A single `Proxy` may therefore carry both `bodies` and `particles`; both mappings are synchronized, velocity-rewound, and harvested around the same destination solve.

Example usage:

```
from newton.solvers.coupled_experimental import SolverCoupled, SolverProxyCoupled

rigid = SolverCoupled.Entry(
    name="rigid",
    solver=lambda view: SolverMuJoCo(model=view),
    bodies=robot_bodies,
    shapes=robot_shapes,
    joints=robot_joints,
    substeps=4,
)
soft = SolverCoupled.Entry(
    name="soft",
    solver=lambda view: SolverVBD(model=view, iterations=20),
    bodies=cloth_bodies,
```

```
    particles=cloth_particles,
    shapes=cloth_shapes,
)

solver = SolverProxyCoupled(
    model,
    entries=[rigid, soft],
    coupling=SolverProxyCoupled.Config(
        proxies=[
            SolverProxyCoupled.Proxy(
                source="rigid",
                destination="soft",
                bodies=robot_bodies,
                proxy_bodies=robot_proxy_bodies,
                mass_scale=0.25,
                mode="lagged",
            )
        ],
        iterations=4,
    ),
)
```

5.4 Linearized ADMM coupling

```
class SolverAdmmCoupled(SolverCoupled):
    @classmethod
    def register_custom_attributes(cls, builder: nt.ModelBuilder) -> None: ...

    @classmethod
    def add_body_particle_attachment(
        cls,
        builder: nt.ModelBuilder,
        body: int,
        particle: int,
        *,
        body_point: tuple[float, float, float] | wp.vec3 = (0.0, 0.0, 0.0),
        stiffness: float = 1.0e4,
        damping: float = 0.0,
        enabled: bool = True,
    ) -> int: ...

    @dataclass(frozen=True)
    class ContactPair:
        source: str
        destination: str
        contact_distance: float | None = None
        detection_margin: float | None = None
```

```
    @dataclass(frozen=True)
    class Config:
        iterations: int = 5
        rho: float = 1.0
        gamma: float = 0.0
        baumgarte: float = 0.0
        joint_stiffness: float = 1.0e4
        joint_damping: float = 0.0
        joint_angular_stiffness: float = 1.0e4
        joint_angular_damping: float = 0.0
        contact_pairs: Sequence["SolverAdmmCoupled.ContactPair"] = ()

    @classmethod
    def auto_detect_contact_pairs(
        cls,
        entries: Sequence[SolverCoupled.Entry],
        *,
        contact_distance: float | None = None,
        detection_margin: float | None = None,
    ) -> list["SolverAdmmCoupled.ContactPair"]: ...

    def __init__(
        self,
        model: nt.Model,
        entries: Sequence[SolverCoupled.Entry],
        coupling: "SolverAdmmCoupled.Config",
    ) -> None: ...
```

`SolverAdmmCoupled` no longer accepts explicit endpoint-interface records. The ADMM constraints are discovered from the shared model:

- cross-solver model joints become quadratic ADMM attachment rows;
- custom `coupling:body_particle_attachment` rows become quadratic rigid-body to particle attachment rows;
- particle-shape, rigid-rigid, and particle-particle contacts are enabled per interface through `ContactPair` records in `Config.contact_pairs`.

For cross-solver joints, the two connected bodies must be owned by different `SolverCoupled.Entry` objects, and the joint itself must be left unowned by sub-solvers so it is not solved twice. The current generic ADMM joint path maps `JointType.BALL` to translational anchor-coincidence rows and `JointType.FIXED` to translational plus angular rows. `JointType.REVOLUTE` adds angular rows that preserve the hinge axis. Other cross-solver joint types are rejected until their constrained subspace is represented exactly in the ADMM local solve. The scalar `joint_stiffness`, `joint_damping`, `joint_angular_stiffness`, and `joint_angular_damping` parameters are the quadratic local-solve stiffnesses and damping coefficients for those model-derived rows. For `JointType.BALL` and `JointType.REVOLUTE`, nonzero `model.joint_friction` entries create angular dry-friction rows. The rows measure relative angular velocity in the appropriate joint frame and apply box maximum-dissipation solves matching the stored joint friction DOFs.

`SolverAdmmCoupled.add_body_particle_attachment` is the convenience path for deformable-to-rigid attachments that cannot be represented by a model joint because one endpoint is a particle.

It registers a model custom frequency `coupling:body_particle_attachment` with SoA attributes for the body index, particle index, body-local point, stiffness, damping, and enabled flag, then appends one row. During construction, rows whose body and particle endpoints are owned by different entries become quadratic ADMM attachment rows. Rows with unowned or same-entry endpoints are ignored, matching the model-joint path's "only cross-solver constraints are owned by the coupler" rule. The helper is equivalent to calling `register_custom_attributes` and filling the custom values directly, so USD or importer paths can author the same attributes without using the Python helper.

Model-derived ADMM contacts are enabled by adding one or more `SolverAdmmCoupled.ContactPair` records to `SolverAdmmCoupled.Config.contact_pairs`. The helper `SolverAdmmCoupled.auto_detect_contact_p ...)` returns one pair for every distinct entry combination, matching the original auto-discovery behavior while keeping contact interfaces explicit in the public configuration. The coupler owns a private collision pipeline for shape-based contact detection and builds contact groups from solver-entry ownership: particle-shape rows are generated between particle entries and shapes on bodies owned by another entry whose shape flags include particle collision; rigid-rigid rows are generated from cross-entry shape contact pairs; and particle-particle rows are generated for cross-entry particle sets through a private hash-grid pass. When a pair's `contact_distance` is `None`, rigid contacts use collision margins, particle-shape contacts use particle radii, and particle-particle contacts use radius sums. Otherwise `contact_distance` is the admissible signed gap for generated rows. Friction is read from model material properties such as `shape_material_mu` and `Model.particle_mu`; it is not a `ContactPair` field. The hash-grid contact stream is an internal fixed-capacity record used only to refresh persistent ADMM rows and report normal force/impulse inside the coupler. Contact paths keep fixed-capacity device arrays and warm-start `u` / `lambda` by stable contact keys when contacts persist from one step to the next.

A possible future extension is to make contact streams the input contract for proxy momentum harvesting itself: stream rows would encode source/proxy endpoint pairs, the proxy step would snapshot destination velocities, and the harvester would write feedback directly through those rows. That design is different from mirroring the current dense momentum-harvest result into streams after the harvest has already happened. The private detection contacts are not forwarded to sub-solvers; callers may still pass their own `Contacts` buffer for ordinary solver contacts. `gamma` controls the proximal term: when it is positive, `SolverAdmmCoupled` scales masses in each `ModelView`, notifies sub-solvers that inertial properties changed, and shifts the begin-of-iteration velocities toward the previous ADMM iterate.

Model-joint attachment usage:

```
link_joint = builder.add_joint_revolute(parent=-1, child=link)
payload_joint = builder.add_joint_ball(
    parent=link,
    child=payload,
    parent_xform=wp.transform(p=wp.vec3(link_hx, 0.0, 0.0)),
    child_xform=wp.transform(p=wp.vec3(-payload_hx, 0.0, 0.0)),
)

rigid = SolverCoupled.Entry(
    name="mjc",
    solver=lambda view: SolverMuJoCo(model=view),
    bodies=[link],
    joints=[link_joint],      # sub-solver-owned world joint
```

```
)
soft = SolverCoupled.Entry(
    name="vbd",
    solver=lambda view: SolverVBD(model=view, iterations=20),
    bodies=[payload],         # payload_joint is intentionally unowned
)

solver = SolverAdmmCoupled(
    model,
    entries=[rigid, soft],
    coupling=SolverAdmmCoupled.Config(
        iterations=8,
        rho=1.0,
        gamma=0.1,
        baumgarte=0.2,
        joint_stiffness=1.0e4,
        joint_damping=10.0,
    ),
)
```

Body-particle attachment usage:

```
ball = builder.add_body(...)
center_particle = particle_start + center_j * (dim_x + 1) + center_i

SolverAdmmCoupled.add_body_particle_attachment(
    builder,
    body=ball,
    particle=center_particle,
    body_point=(0.0, 0.0, 0.0),
    stiffness=50.0,
    damping=1.0,
)

solver = SolverAdmmCoupled(
    model,
    entries=[
        SolverCoupled.Entry(
            name="mjc",
            solver=lambda view: SolverMuJoCo(model=view),
            bodies=[ball],
        ),
        SolverCoupled.Entry(
            name="vbd",
            solver=lambda view: SolverVBD(model=view, iterations=20),
            particles=list(range(model.particle_count)),
        ),
    ],
    coupling=SolverAdmmCoupled.Config(iterations=8, rho=50.0),
```

```
)
```

Collision-detected particle-shape contact usage:

```
solver = SolverAdmmCoupled(
    model,
    entries=[drop, tray],
    coupling=SolverAdmmCoupled.Config(
        iterations=12,
        rho=45.0,
        baumgarte=0.1,
        contact_pairs=[
            SolverAdmmCoupled.ContactPair(
                source="drop",
                destination="tray",
                contact_distance=0.04,
                detection_margin=0.08,
            )
        ],
    ),
)
```

Collision-detected rigid-rigid contact usage:

```
solver = SolverAdmmCoupled(
    model,
    entries=[box_entry, plane_entry],
    coupling=SolverAdmmCoupled.Config(
        iterations=30,
        rho=5.0,
        gamma=0.2,
        baumgarte=0.03,
        contact_pairs=[
            SolverAdmmCoupled.ContactPair(
                source=box_entry.name,
                destination=plane_entry.name,
            )
        ],
    ),
)
```

Collision-detected particle-particle contact usage:

```
solver = SolverAdmmCoupled(
    model,
    entries=[cloth_a, cloth_b],
    coupling=SolverAdmmCoupled.Config(
        iterations=10,
        rho=30.0,
        baumgarte=0.2,
```

```
        contact_pairs=SolverAdmmCoupled.auto_detect_contact_pairs(
            [cloth_a, cloth_b],
            detection_margin=0.02,
        ),
    ),
)
```

5.5 No single-pair convenience wrappers

The implementation intentionally avoids exported MuJoCo+VBD, MuJoCo+MPM, or MuJoCo+VBD-ADMM wrapper classes. The examples construct `SolverProxyCoupled`, `SolverAdmmCoupled`, or the base `SolverCoupled` directly. Example-only sharing was also removed: each example now carries its own rigid-solver selection and graph-capture setup so the framework behavior is visible at the construction site. MPM proxy examples that use deformable colliders rely on `SolverImplicitMPM`'s default constructor-time collider setup against the entry `ModelView`, so proxy mass relaxation is visible without an extra example-side setup call. The XPBD-MPM particle example instead exercises transfer-active pure proxy particles, with no triangles and no collider-particle registration. This keeps the default coupler path under test for the common two-solver cases and makes any missing generic hook visible.

5.6 Optional `CouplingInterface` hooks

The implemented coupling hooks keep `step()` source-compatible. A solver inherits `CouplingInterface` when it participates in coupled simulations. Omitted hook methods default to the model/state fallback path; hooks listed in `coupling_unsupported` reject the coupling mode explicitly. The hook enums are nested under `CouplingInterface` so the experimental `newton.solvers.coupled_experimental` namespace stays compact rather than growing a family of top-level helper symbols.

```
class CouplingInterface:
    class EndpointKind(IntEnum):
        BODY = 0
        PARTICLE = 1

    class InputStateFlags(IntFlag):
        BODY_Q = 1 << 0
        BODY_QD = 1 << 1
        PARTICLE_Q = 1 << 2
        PARTICLE_QD = 1 << 3
        JOINT_Q = 1 << 4
        JOINT_QD = 1 << 5
        BODY_F = 1 << 6
        PARTICLE_F = 1 << 7
        JOINT_F = 1 << 8

        BODY = BODY_Q | BODY_QD
        PARTICLE = PARTICLE_Q | PARTICLE_QD
        JOINT = JOINT_Q | JOINT_QD
        FORCE = BODY_F | PARTICLE_F | JOINT_F
        ALL = BODY | PARTICLE | JOINT | FORCE
```

```
class Hook(IntFlag):
    BODY_PROXY_REWIND_VELOCITY = 1 << 0
    PARTICLE_PROXY_REWIND_VELOCITY = 1 << 1
    BODY_PROXY_HARVEST = 1 << 2
    PARTICLE_PROXY_HARVEST = 1 << 3
    EFFECTIVE_MASS_DIAGONAL = 1 << 4
    EFFECTIVE_MASS_BLOCK = 1 << 5
    NOTIFY_INPUT_STATE_UPDATE = 1 << 6
    PROXY_CONTACT_PREPARE = 1 << 7

coupling_unsupported: ClassVar[frozenset[Hook]] = frozenset()

def coupling_eval_effective_mass(
    self,
    endpoint_kind: wp.array[int],
    endpoint_index: wp.array[int],
    endpoint_local_pos: wp.array[wp.vec3],
    out: wp.array[float],
) -> None: ...

def coupling_eval_effective_mass_block(
    self,
    endpoint_kind: wp.array[int],
    endpoint_index: wp.array[int],
    endpoint_local_pos: wp.array[wp.vec3],
    out_mass: wp.array[float],
    out_inertia: wp.array[wp.mat33] | None = None,
) -> None: ...

def coupling_notify_input_state_update(
    self,
    state: nt.State,
    flags: CouplingInterface.InputStateFlags | int,
    *,
    restart: bool = False,
    dt: float = 0.0,
) -> None: ...

def coupling_harvest_proxy_wrenches(
    self,
    body_local_to_proxy_global: wp.array[int],
    out_body_f: wp.array[wp.spatial_vector],
    *,
    state: nt.State | None = None,
    state_out: nt.State | None = None,
    contacts: nt.Contacts | None = None,
    dt: float = 0.0,
```

```
) -> None: ...

def coupling_rewind_proxy_body_velocity(
    self,
    body_local_to_proxy_global: wp.array[int],
    state: nt.State,
    coupling_forces: wp.array[wp.spatial_vector],
    dt: float,
) -> None: ...

def coupling_prepare_proxy_contacts(
    self,
    state: nt.State,
    contacts: nt.Contacts | None,
    *,
    contacts_freshly_detected: bool = False,
) -> nt.Contacts | None: ...

def coupling_harvest_proxy_particle_forces(
    self,
    particle_local_to_proxy_global: wp.array[int],
    out_particle_f: wp.array[wp.vec3],
    *,
    state: nt.State | None = None,
    state_out: nt.State | None = None,
    contacts: nt.Contacts | None = None,
    dt: float = 0.0,
) -> None: ...

def coupling_rewind_proxy_particle_velocity(
    self,
    particle_local_to_proxy_global: wp.array[int],
    state: nt.State,
    coupling_forces: wp.array[wp.vec3],
    dt: float,
) -> None: ...
```

`CouplingInterface` is a marker and enum container. The method signatures above are the hook names recognized by the coupled wrappers. If a solver implements one of those methods, the wrapper calls it; if it omits the method, the wrapper uses the public model/state fallback path unless the hook is listed in `coupling_unsupported`. Body and particle coupling forces are written directly into `state.body_f` and `state.particle_f`; force injection is no longer a solver hook. The maps used by the couplers are dense local-to-global index maps. For example, the default body path loops over local body ids `b` and, when `body_local_to_global[b] >= 0`, adds `body_f[body_local_to_global[b]]` into `state.body_f[b]`. Normal ADMM force buffers use the identity map. Proxy feedback uses a source-local-to-global-proxy map, so harvested forces stored at the global proxy id are applied to the source solver's local id without changing the indexing

convention of the feedback buffer.

`restart=True` is a contextual argument on `coupling_notify_input_state_update`. ADMM and proxy coupling pass it when they restart a solver input state for another relaxation pass over the same top-level time step. Solvers with private step history can then realign that history with the public input state without treating the reset as a physical teleport. VBD uses this to reset rigid `body_q_prev` on repeated coupling iterations, while ordinary proxy pose updates still use the proxy velocity-target path.

Model-view mass and inertia overrides are also not coupling hooks. Couplers mutate the destination `ModelView` directly: body proxy virtual inertia uses `set_body_inertial_properties`, and particle proxy virtual mass uses `set_particle_mass`. After such post-construction mutations the coupler calls the ordinary solver `notify_model_changed` path, using `BODY_INERTIAL_PROPERTIES` for body mass/inertia changes and a model properties notification for particle mass changes. Solvers with private model caches should therefore handle these changes in `notify_model_changed` rather than through a coupling-specific override hook. ADMM uses the same `ModelView` mutation mechanism for the proximal inertia term: when `gamma>0`, it scales owned body and particle masses by `1+gamma` before sub-solver construction and notifies solvers after construction.

`NOTIFY_INPUT_STATE_UPDATE` is the generic state-side notification hook. The fallback is no extra action because the coupler has already written the updated values directly into the public `State`. Custom solvers use this hook to synchronize private input buffers or to translate public state updates into solver-native history. `SolverProxyCoupled` calls it after state distribution, source force-buffer updates, source-to-proxy synchronization, and lagged proxy velocity rewinds. `SolverAdmmCoupled` calls it at the beginning of each ADMM iteration after restoring the iteration state, and after writing the proximal velocity target `(v_n + gamma v_k)/(1+gamma)` into `body_qd`, `particle_qd`, and `joint_qd`. There is therefore no separate ADMM-specific proximal-inertia or proximal-velocity hook; proximal inertia uses model-view mass scaling, and proximal velocity uses this generic state-update notification hook.

`BODY_PROXY_REWIND_VELOCITY` and `PARTICLE_PROXY_REWIND_VELOCITY` let a destination solver rewind synchronized proxy velocities before its step. MPM uses these hooks for collider velocity preparation because its body and deformable-particle colliders are prescribed rather than integrated from public force buffers. For pure transfer-active particle proxies, the same MPM hook also subtracts gravity from the lagged input velocity so gravity is not harvested as coupling feedback. Without custom capabilities, `SolverProxyCoupled` applies the generic lagged velocity rewind from the destination `ModelView`, subtracting previously applied feedback, public force inputs, and gravity. VBD instead uses `NOTIFY_INPUT_STATE_UPDATE` for smooth proxy-body teleport handling and the generic rewind for lagged feedback. `BODY_PROXY_HARVEST` and `PARTICLE_PROXY_HARVEST` let a solver report feedback from private buffers or impulses; otherwise proxy feedback is estimated from destination proxy momentum change. For body and particle proxies, `state` is the prepared destination input state, `state_out` is the destination output state after the proxy solve, and `contacts` is the contact set passed to that solve. This lets custom harvesters report contact-only feedback at the accepted configuration. VBD uses this to avoid harvesting the final AVBD body force buffer, which may include non-contact terms or only the last nonlinear force assembly, and instead reduces the final rigid-rigid and body-particle contact forces onto proxy bodies. Proxy rewind and harvest hooks receive a dense local-to-global-proxy map. The local id is a body or particle index in the destination solver's `ModelView`; proxy entries map to the corresponding global proxy id in the parent model, and non-proxy entries are `-1`. State arrays and solver-private buffers remain local-indexed. The `coupling_forces`, `out_body_f`, and `out_particle_f` buffers are indexed by global proxy ids. `SolverProxyCoupled` maps the proxy-indexed feedback back to source local ids through the public force-buffer path when injecting it into the driving solver. `EFFECTIVE_MASS_DIAGONAL`

and `EFFECTIVE_MASS_BLOCK` are query hooks, not mutation hooks: they let couplers ask a solver for endpoint effective mass. Endpoint batches are passed as structure-of-arrays Warp buffers: `endpoint_kind`, `endpoint_index`, and `endpoint_local_pos`. The arrays all have the same length as the output mass buffer. `endpoint_kind` stores body/particle kind values, `endpoint_index` stores the local model-view body or particle id, and `endpoint_local_pos` stores body-frame points for body endpoints (`0` for particles). The diagonal hook defaults to model mass/inertia. The block hook defaults to model mass and full body inertia, or to the diagonal/model fallback only when the caller can accept a scalarized body estimate. Proxy coupling uses the block hook for full virtual body inertia and the diagonal hook for particle proxy mass; ADMM uses diagonal endpoint masses for interface/contact weighting and falls back to model masses when no custom query is supplied.

`coupling_prepare_proxy_contacts` is called for body-proxy destination solves so the destination contact set can filter proxy contacts outside its ownership/feedback boundary. Proxy-local collision generation is configured on `SolverProxyCoupled.Proxy` rather than through this hook; the hook receives `contacts_freshly_detected=True` only when that proxy collision pipeline has just run. VBD uses this to filter newly detected proxy contacts and to call `set_rigid_history_update` on the same cadence as contact refreshes. XPBD additionally treats view-local `ParticleFlags.PROXY` particles as contact-only proxies, filtering proxy-proxy and proxy-static particle contacts while retaining owned-particle versus proxy contact.

6 Comparison

Variant	Interface solve	API cost	Main tradeoff
Collider exchange	independent kinematic-contact solves	low	easiest to stage, but action-reaction and complementarity are not solved as one interface problem
One-way	one side prescribes motion, the other solves contact	low	controlled when one side is effectively infinite mass; otherwise intentionally asymmetric
Proxy / virtual inertia	destination solver solves contacts against finite-mass replicas	medium	reuses a native contact solver, but stability depends on proxy inertia and force feedback is lagged unless fixed-point iterations are used
Linearized ADMM	external-force sub-solver sweeps plus local interface proximal step	medium-high	symmetric in ownership and gives residuals/dual impulses, but needs in-step iterations and proximal tuning
Implicit-interface ADMM	sub-solvers include implicit interface quadratic terms	high	fewer ADMM iterations and less proximal tuning, but requires implicit attachment or Hessian support in each sub-solver

The schemes therefore occupy different points on an implementation spectrum. Collider exchange and one-way coupling minimize new API surface. Proxy coupling adds replica metadata

and force harvesting in exchange for reusing one solver's contact machinery. Linearized ADMM adds a shared interface iteration without requiring arbitrary Hessians inside sub-solvers. Implicit-interface ADMM asks for the richest solver API and is the closest split form to a monolithic coupled solve.